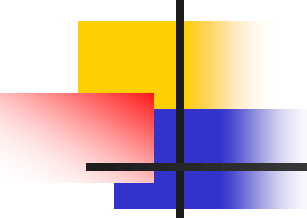


ECSE 4320/6320 Advanced Computer Systems

Thread 2: How to Make Computing Cheaper or
More Reliable?



Make Computing Cheaper

- ❑ Reduce the hardware component cost

- Technology scaling

- \$/transistor no longer reduces, would scaling still help to make computing cheaper?

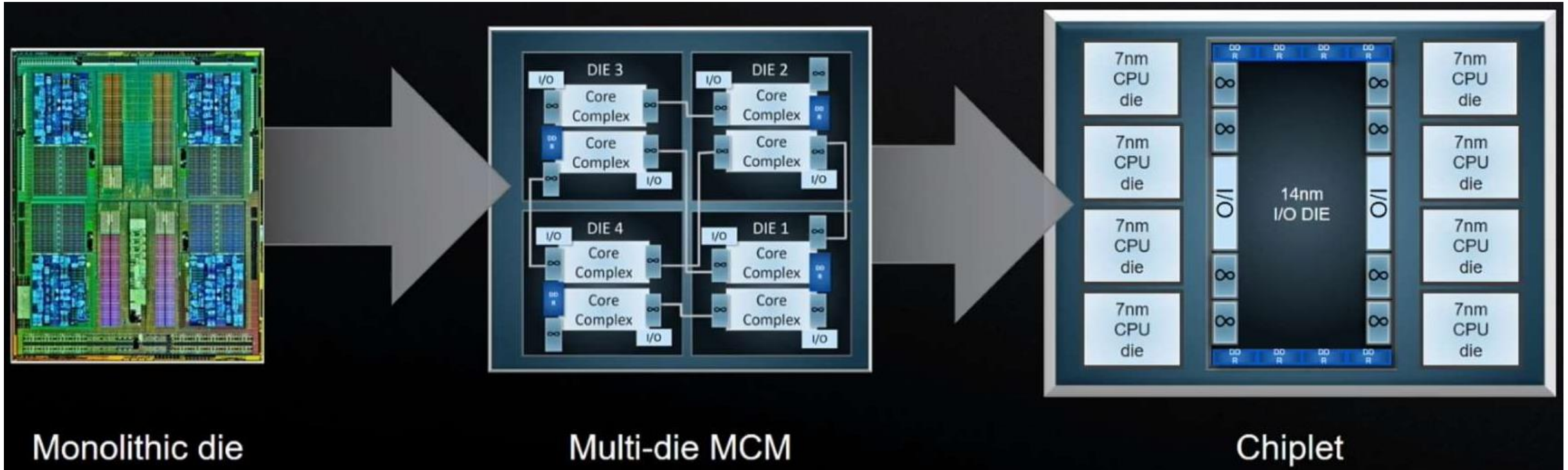
- New technologies

- ❑ Reduce the software development cost

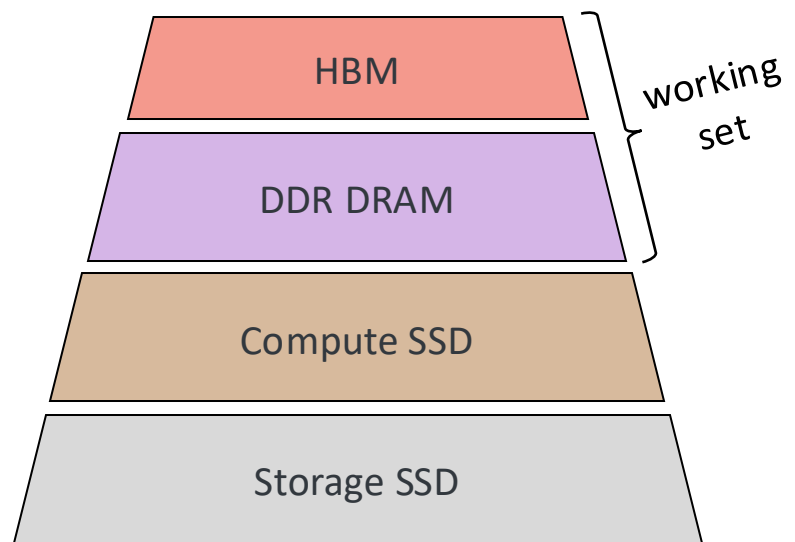
- ❑ Improve the hardware utilization efficiency

Lower the Cost: Heterogeneous Integration

Chiplet in AMD CPU

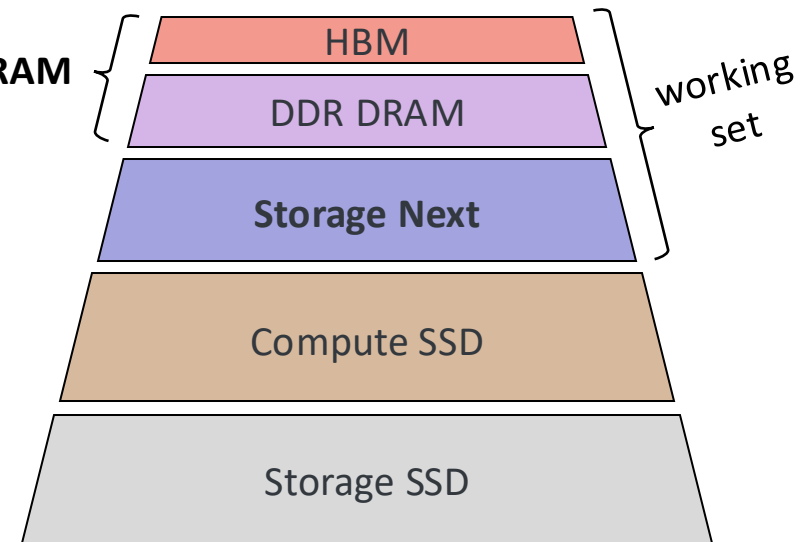


Storage Next

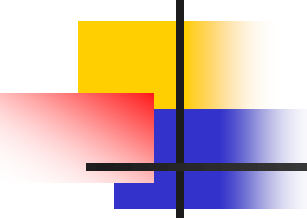


- ✗ NAND = Second-Class
- ✗ CPU-driven I/O
- ✗ Heavy DRAM reliance

>3x less DRAM

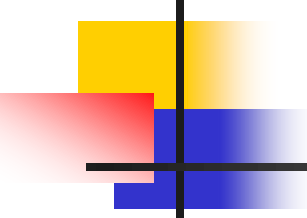


- ✓ NAND = First-Class
- ✓ GPU = New I/O Engine
- ✓ Less DRAM, Same Throughput



Lower the Cost: Data Compression

- ☐ Reduce the memory/storage cost
- ☐ Reduce the data transfer cost
- ☐ How widely is data compression used in practice?



Data Centers

- ❑ The backbone of modern information technology infrastructure
 - AWS, Microsoft Azure, Google, Alibaba, Tencent, Meta, ...
- ❑ Inside one data center
 - 1000's servers with EB data + power converter (even generator) + cooling

Amortized Cost*	Component	Sub-Components
~45%	Servers	CPU, memory, disk
~25%	Power infrastructure	UPS, cooling, power distribution
~15%	Power draw	Electrical utility costs
~15%	Network	Switches, links, transit

"The Cost of a Cloud: Research Problems in Data Center Networks" Greenberg, Hamilton, Maltz, Patel, 2009

*3-year amortization for servers, 15-year for infrastructure

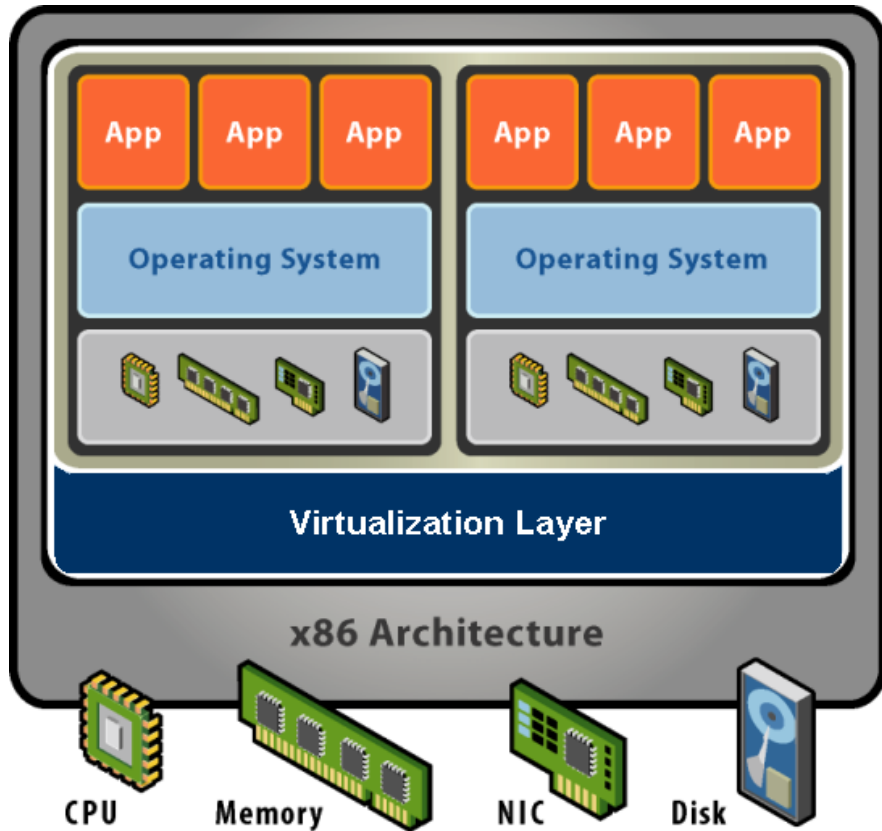
Server Cost

30% utilization is already considered “good” in data centers

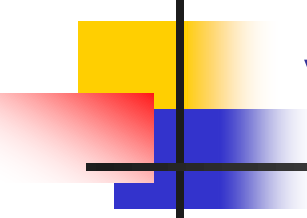


Source: Counterpoint Research – AI 360 Research

Virtualization: Hardware Support

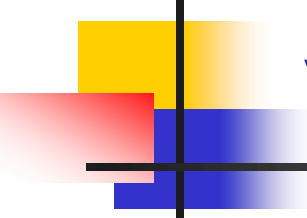


- ❑ Hardware-level Abstraction
 - Behaves like hardware
 - Encapsulates all OS and application state
- ❑ Virtualization Layer
 - Extra level of indirection
 - Decouples hardware, OS
 - Enforces isolation
 - Multiplexes physical hardware across VMs



Virtualization: Hardware Support

- ❑ A virtual machine monitor (VMM aka hypervisor) is a kernel where OSES are 'applications'
 - Can abstract memory (guest physical addresses to machine physical addresses)
 - Multiplex between OSES and trap on specified exceptions/violations
 - Provide a consistent & abstracted view of hardware



Virtualization: Hardware Support

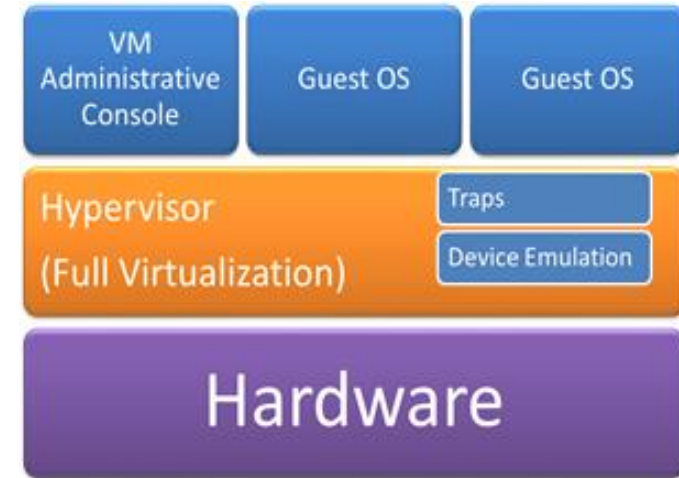
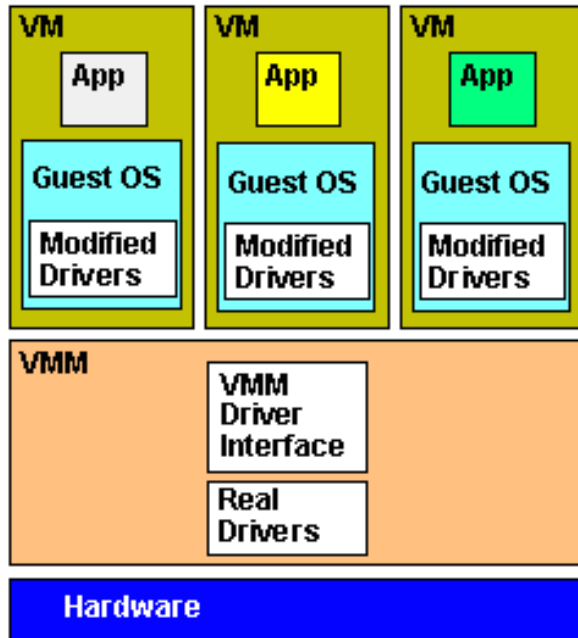
- ❑ Desirable properties of virtualization

- Equivalent execution: Programs running in the virtualized environment run identically to running natively
- Performance: A statistically dominant subset of the instructions must be executed directly on the CPU
- Safety and isolation: A virtual machine monitor must completely control system resources

Virtualization: Hardware Support

1. Full Virtualization

- Everything is virtualized
- Full hardware emulation

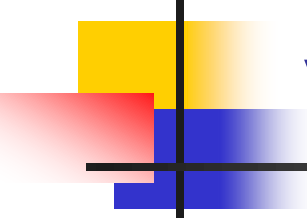


2. Paravirtualization

- Guest OS is virtualization aware (e.g., disk driver talks to VMM directly)

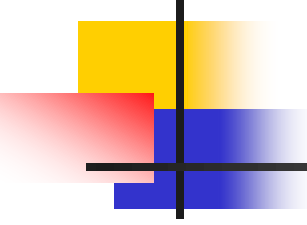
3. Hardware-assisted virtualization

- Server hardware is virtualization aware
- Removes CPU emulation bottleneck



Virtualization: x86 Support

- ❑ x86 virtualization extension (AMD-V, Intel VT-x)
 - Add new instructions that allow guest OS perceives itself as running with full privilege (ring 0), but the host OS remains protected
 - Eliminate need for guest OS binary patching / translation
 - Facilitate support for Legacy OS
 - Eliminate need for complex SW techniques → simpler and smaller VMMs
 - Fewer unwanted Guest & VMM transitions → improves performance

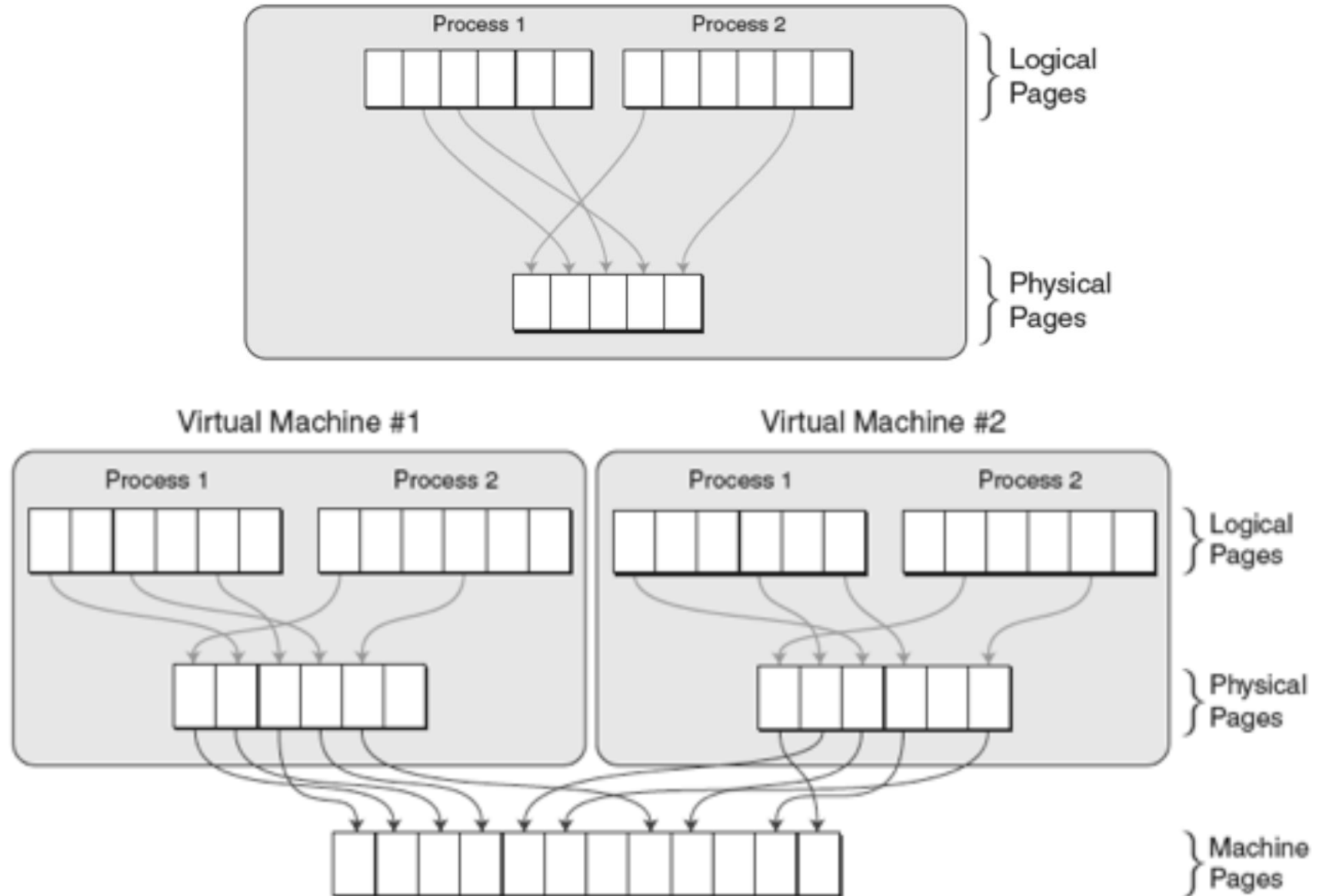


Memory Management Virtualization

- ❑ A quick review of memory management
 - OS exposes virtual memory addresses and handles virtual-physical address mapping
 - CPU contains translation lookaside buffer (TLB) to assist virtual-physical address mapping
 - CPU handles TLB misses in HW
 - ❖ One control register (CR3) points to the page table root
 - ❖ HW walks the page tables

Memory Management Virtualization

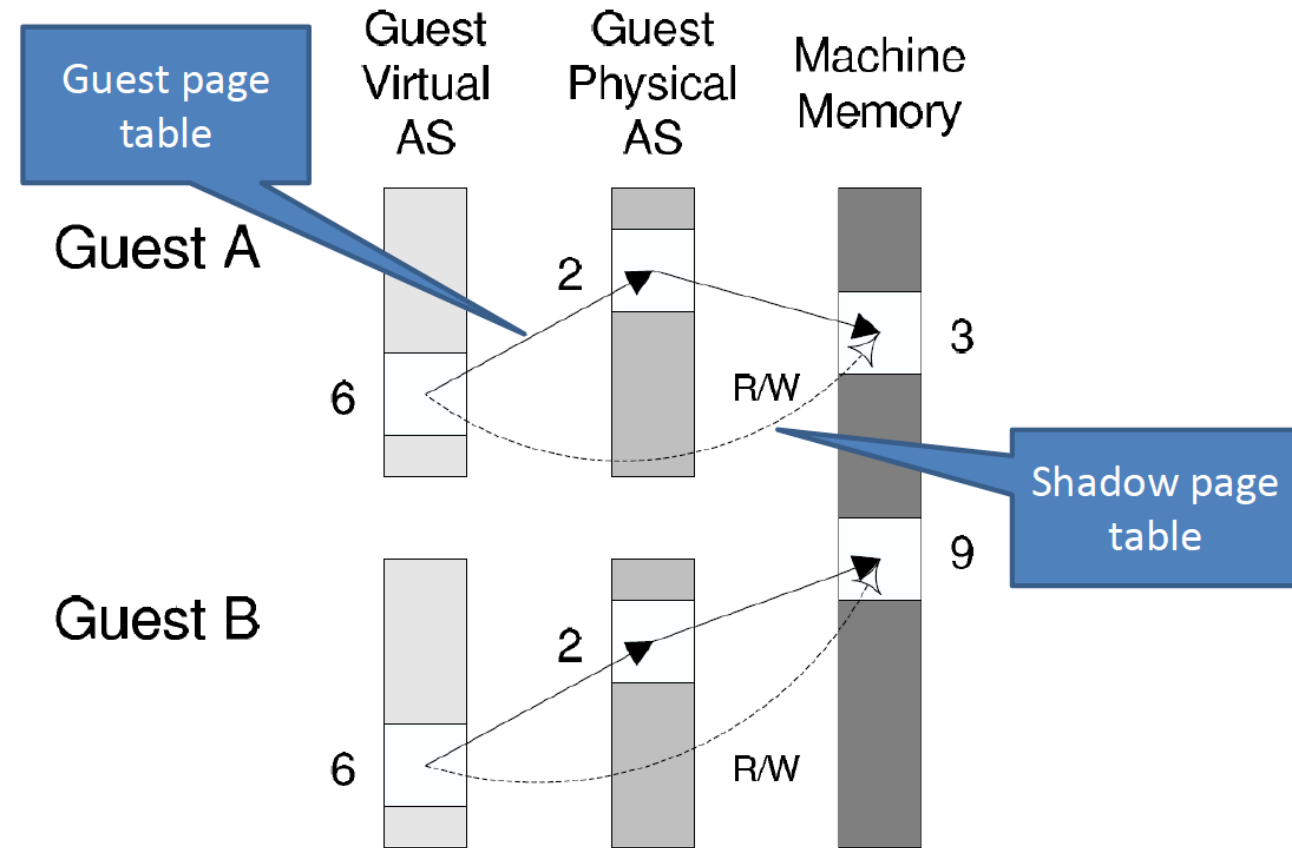
- ❑ Native vs. virtualized



Memory Management Virtualization

❑ Shadow page table

- ✓ Transparent to guest VMs
- ✓ Good performance when working set is stable
- x Big overhead of keeping two page tables consistent



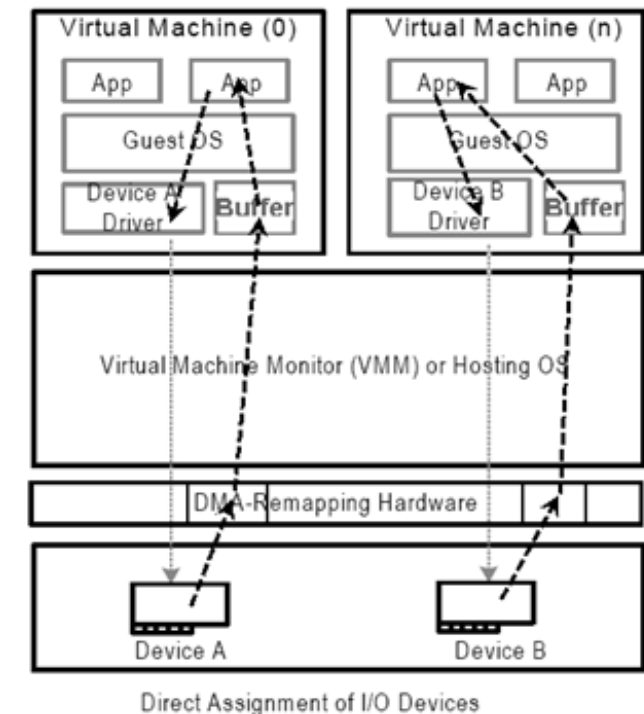
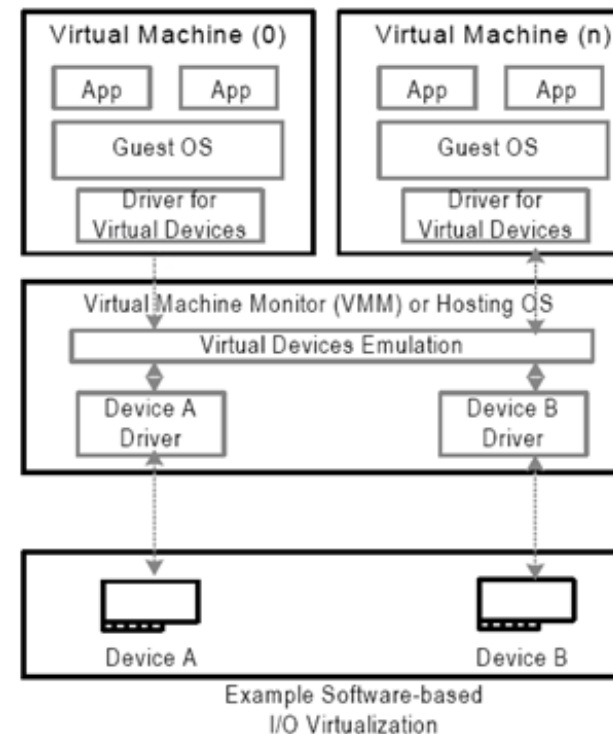
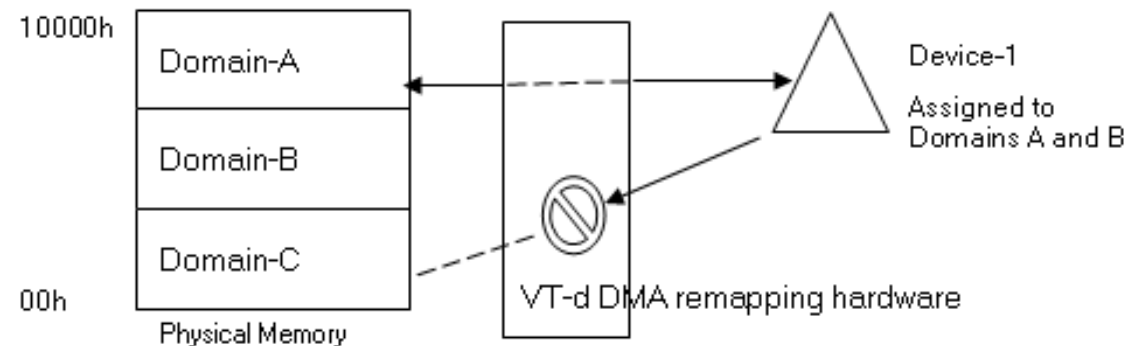
Memory Management Virtualization

- ❑ CPU hardware support: Nested page tables
 - HW keeps a second set of page tables that map from physical to machine addresses
 - On a TLB miss, first find physical address from guest page tables, then map to machine address
- ❑ Intel EPT (Extended Page Table)
- ❑ AMD RVI (Rapid Virtualization Indexing)



IO Virtualization: Hardware Support

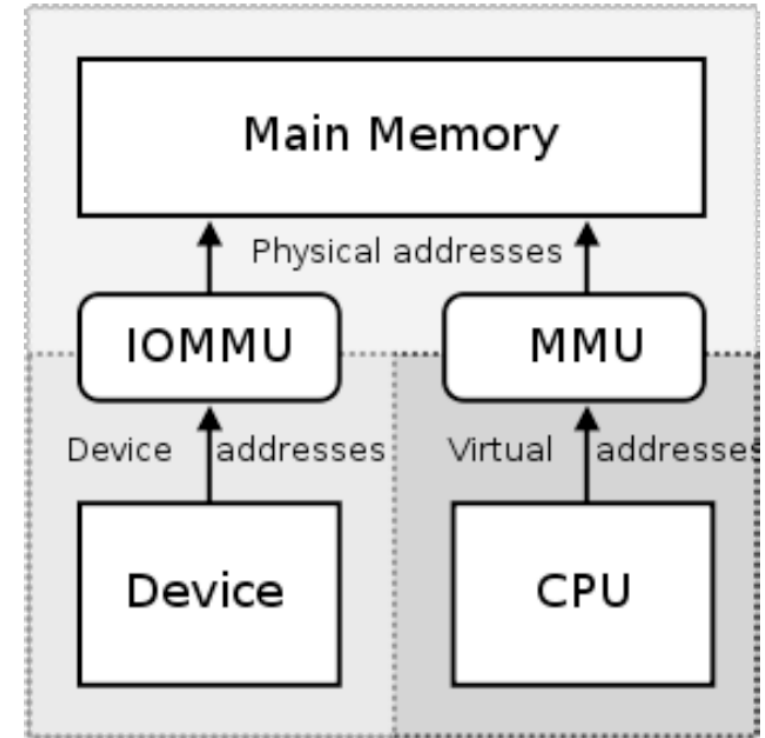
- ❑ Enable protection by restricting DMA of the devices to pre-assigned domains or physical memory regions
- ❑ Enable direct assignment of a device to a VM



IO Virtualization: Hardware Support

❑ Intel Vt-d and AMD IOMMU

- Translates memory addresses from "IO space" to "physical space"
- Improve reliability and security through device isolation using hardware assisted remapping
- Improve I/O performance and availability by direct assignment of devices



IO Virtualization: SR-IOV

- ❑ However, direct I/O is but not scalable: 50~100 virtual machines vs. a few I/O devices?
- ❑ Solution: SR-IOV (single root I/O virtualization)
 - A specification that allows a PCIe device to appear as multiple separate PCIe devices: physical function (PF) and virtual function (VF)
 - PF: The primary function of the device and advertises the device's SR-IOV capabilities
 - VF: Lightweight PCIe functions that contain resources necessary for data movement but have a carefully minimized set of configuration resources

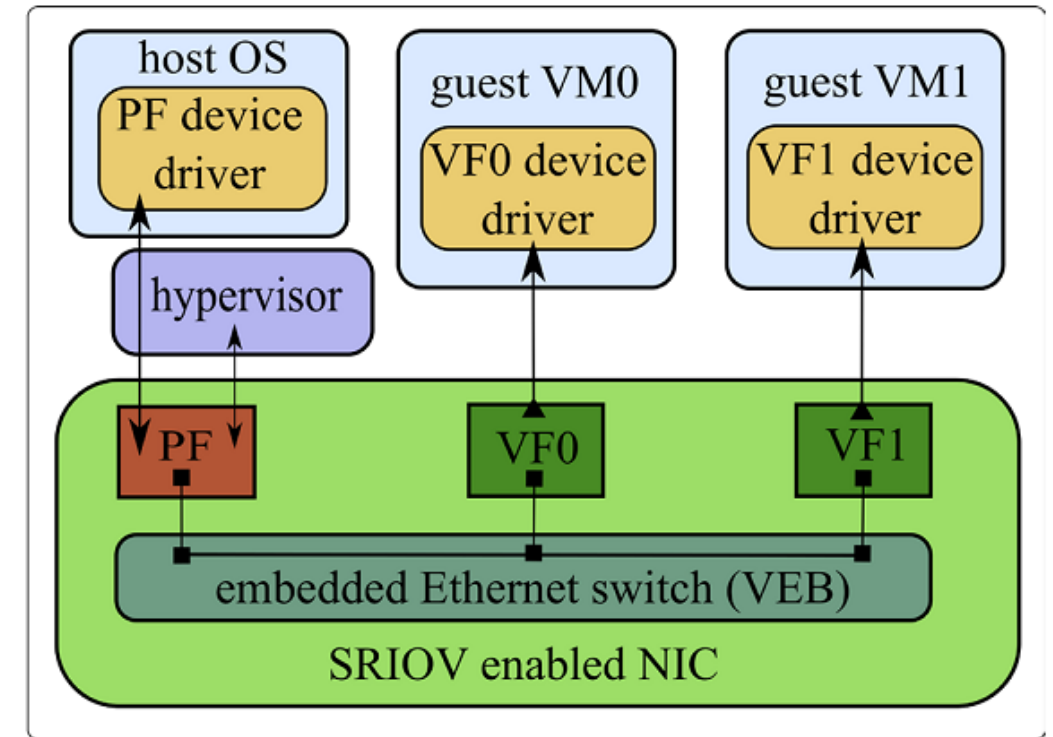
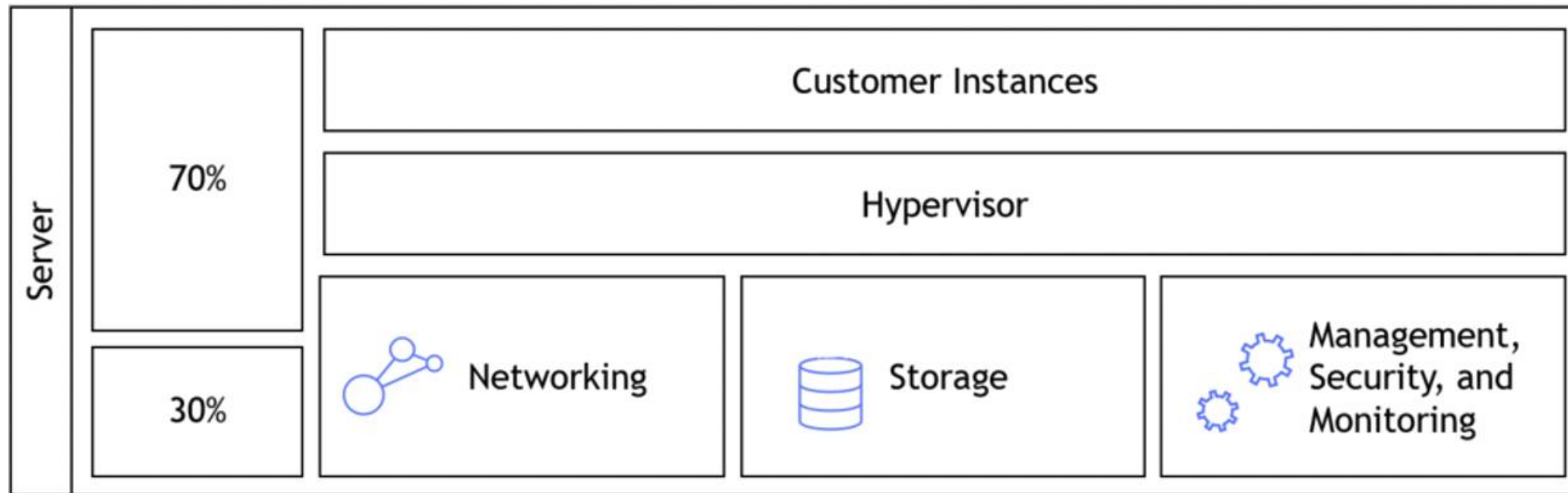


Figure 2: SRIOV NIC in a virtualized environment

Virtualization Hardware Support: One Step Further

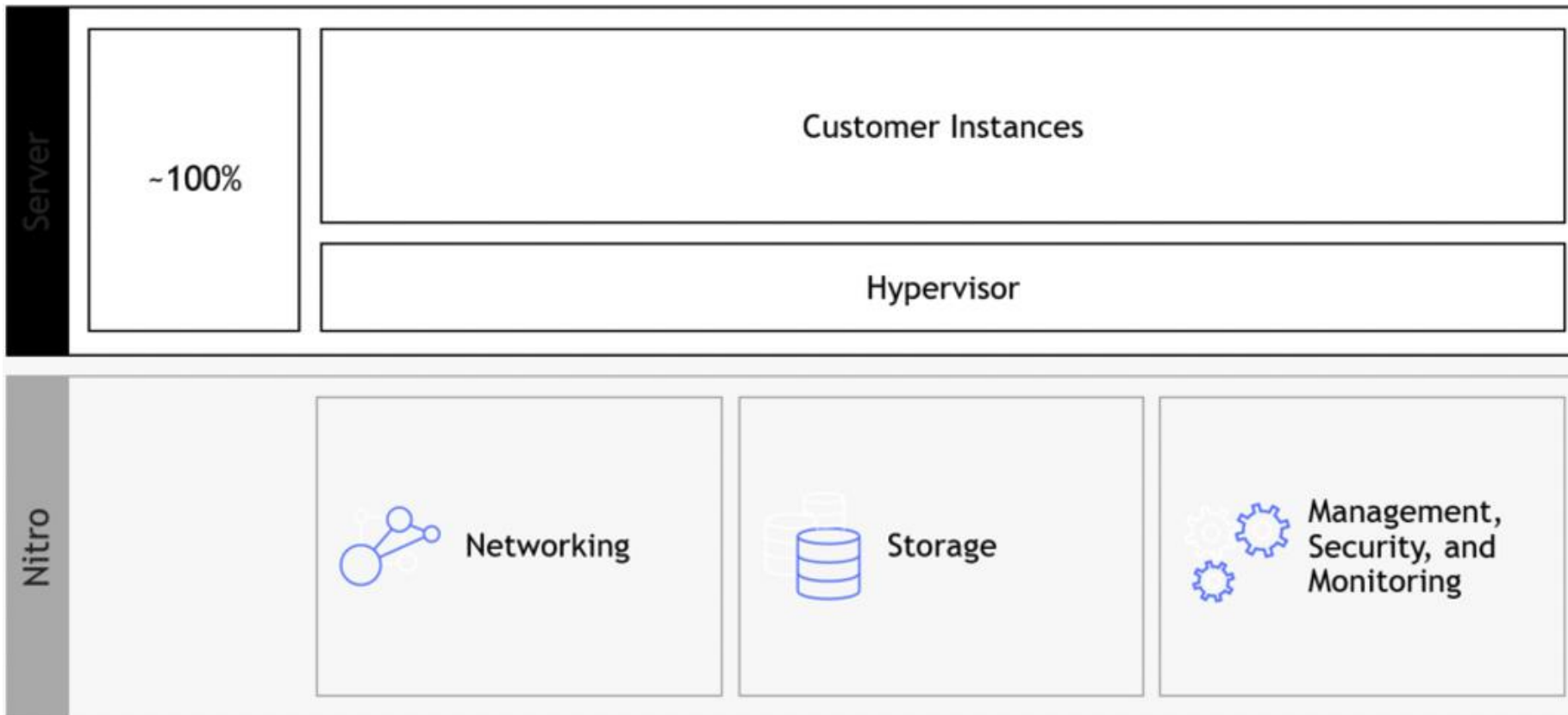
- ❑ Complement CPU with special-purpose hardware (e.g., AWS Nitro)



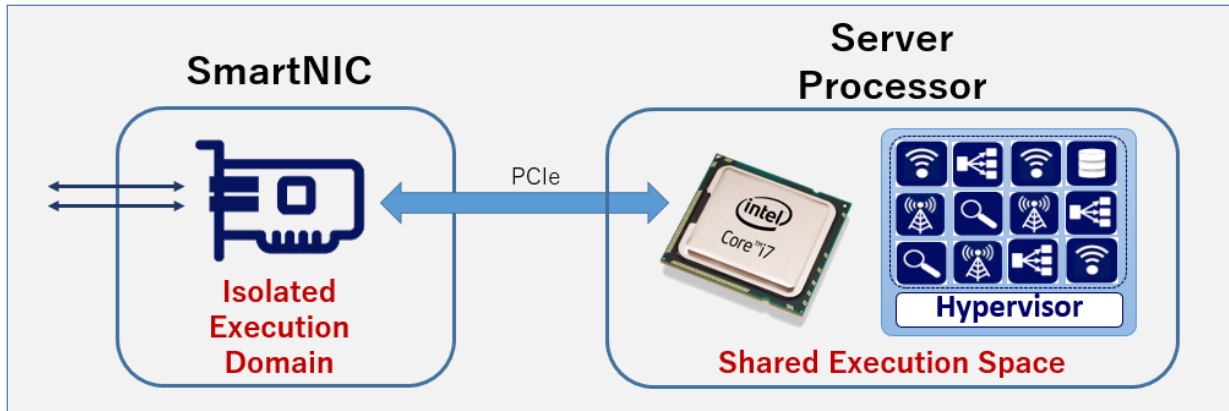
CPU-only platform

Virtualization Hardware Support: One Step Further

- ❑ Complement CPU with special-purpose hardware (e.g., AWS Nitro)



Heterogeneous Computing: SmartNIC



- Network processing (e.g., packet encapsulation, encryption)
- Network virtualization (e.g., SR-IOV)
- Hardware root of trust
- Public/private-key security/encryption

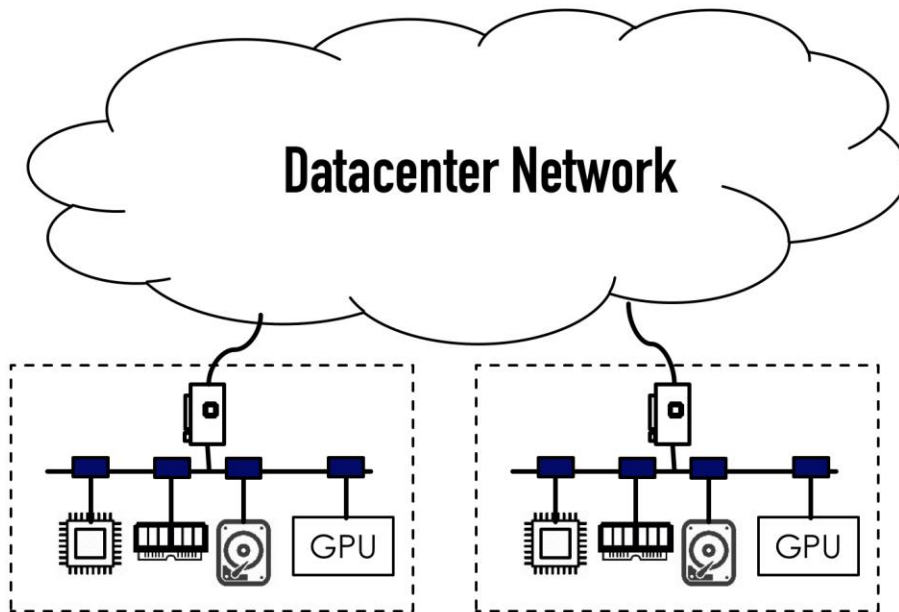
❑ Rationale

- As network bandwidth continues to grow (e.g., 50Gbps → 100Gbps → ...), it becomes increasingly expensive to use host CPUs for handling all the network-related processing
- It is much cheaper (at least 10x) to use a customized SoC (ARM + ASIC) for handling most network-related processing → SmartNIC
- Off-loading network processing from host CPUs also reduces CPU cache pollution and release cache/memory resource to other applications

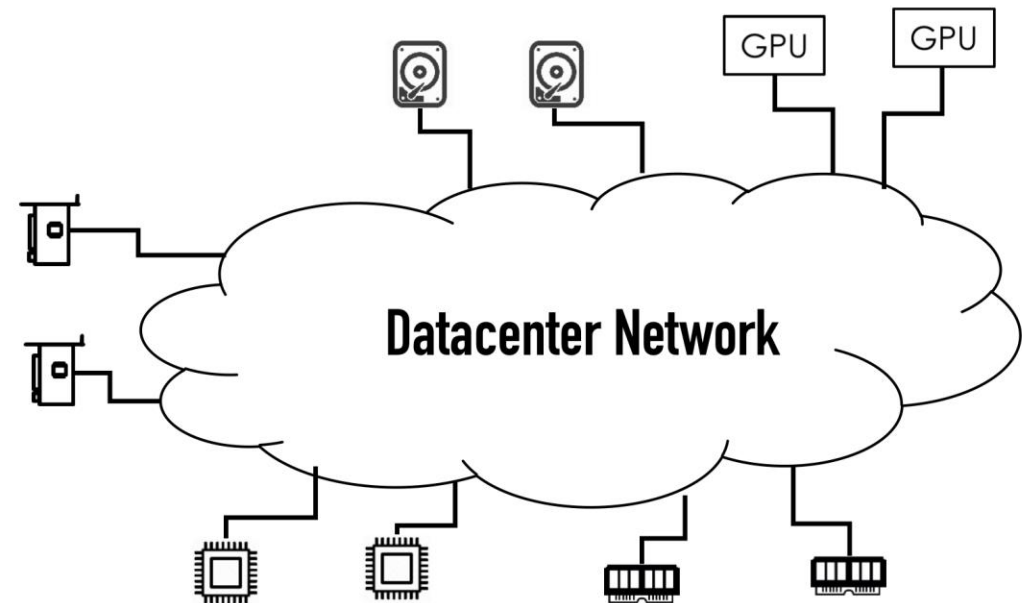
Faster Network: Disaggregated Data Centers

Disaggregated Datacenters

Current Datacenter: Server-Centric

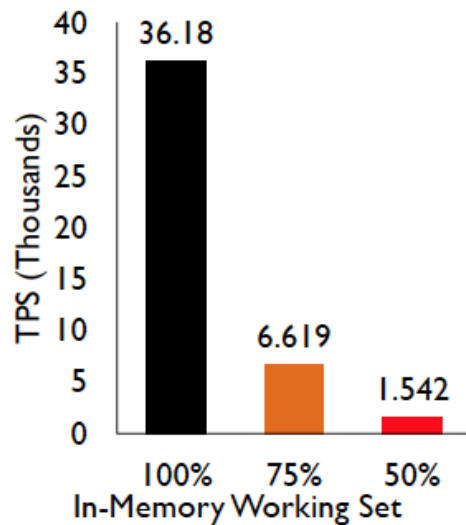


Future datacenter: Disaggregated?

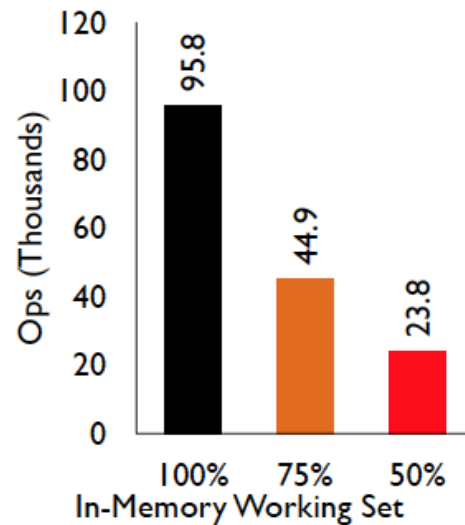


One Step Further: Compute/Memory Disaggregation

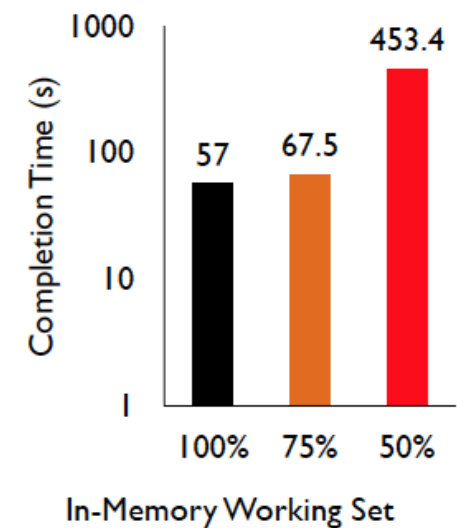
- ❑ Data centers aim to run as many VMs/applications as possible on each server
- ❑ How much memory per server, per VM/application?
 - Under-allocation → severe performance loss
 - Over-allocation → memory underutilization (e.g., 30~50%)



TPC-C on VoltDB



FB Workload on Memcached

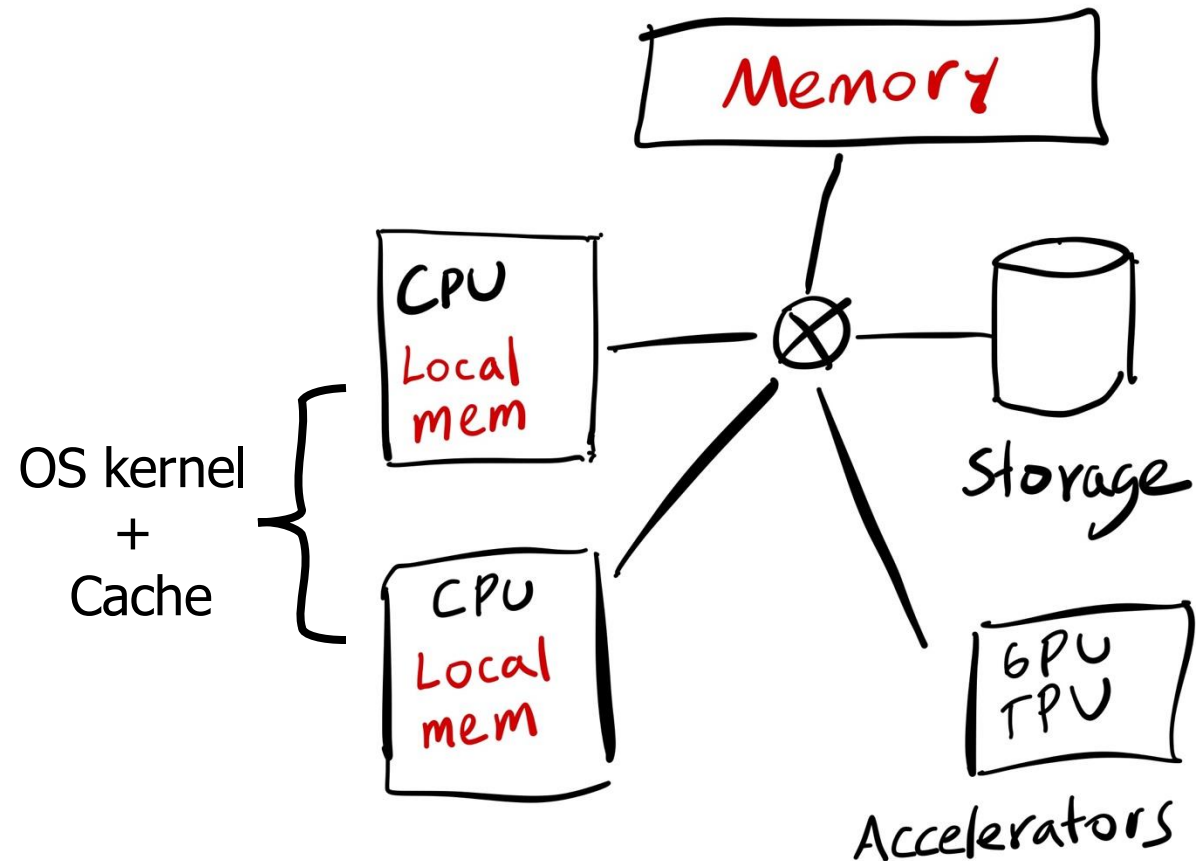


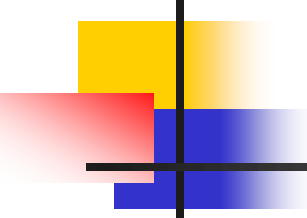
PageRank on PowerGraph

One Step Further: Compute/Memory Disaggregation

❑ Detach memory from compute

- Independence
 - ✓ Evolve independently
 - ✓ Scale independently
 - ✓ Fail separately
- Flexible provisioning
- Less waste



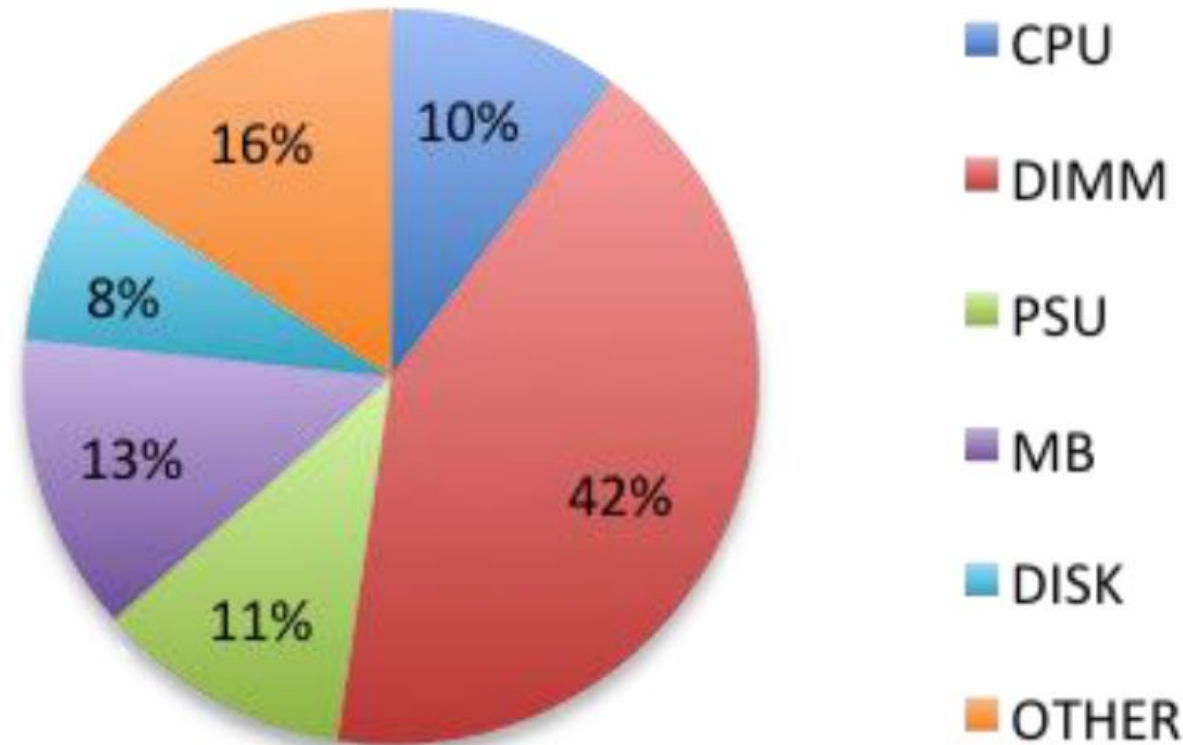


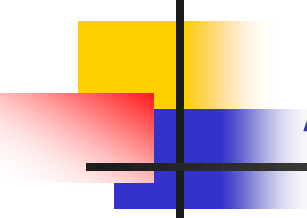
DRAM Reliability

- ❑ DRAM raw reliability tends to degrade with the technology scaling
 - (64+8)-bit SEC-DED being used in computing systems
 - Enterprise-grade computers implement much stronger error/fault tolerance
 - ❖ Reed-Solomon code
 - ❖ RAID
 - DDR5 DRAM chips internally implement (128+8)-bit SEC
- ❑ Spare redundancy (row, column, and array)

A little about RAS

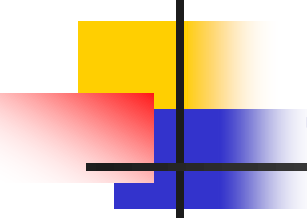
- ❑ RAS: Reliability, Availability, and Serviceability
- ❑ Remember: Hardware failure is no longer a small-probability event
- ❑ Demand close collaboration across silicon hardware, firmware (BIOS), and OS
- ❑ MCA (machine check architecture): Intel/AMD mechanism through which CPU reports hardware errors to the OS





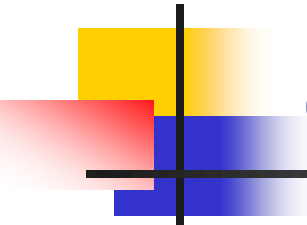
A little about RAS

- ❑ Keyword: **redundancy!**
- ❑ CPU: operate in the lockstep mode to perform master-slave or majority voting
- ❑ Memory: ECC (simple & advanced, system-level & on-chip), CRC, chipkill, scrubbing, sparing, bad page offlining, error logging
- ❑ I/O: CRC
- ❑ Storage: RAID, erasure coding, scrubbing, CRC, error logging
- ❑ Power/cooling: duplicate component, over-design & protection, runtime throttling
- ❑ Systems: host swapping, failure prediction (AI/ML), fault domain, virtual machines, serverless, independent management processor



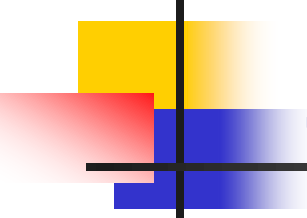
Security in Data Centers

- ❑ In addition to cost, security has become a first-class citizen in data centers
- ❑ Global IT security spending is already over \$100B
- ❑ Security is a very broad and sophisticated area



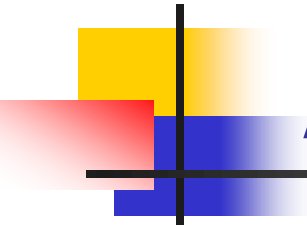
Cryptography Basics

- ❑ Encryption algorithm consists of
 - Set K of keys
 - Set M of Messages
 - Set C of ciphertexts (encrypted messages)
- ❑ Encryption function $E: K \rightarrow (M \rightarrow C)$; Decryption function: $K \rightarrow (C \rightarrow M)$
- ❑ An encryption algorithm must provide this essential property: Given a ciphertext $c \in C$, a computer can compute m such that $E(m) = c$ only if it possesses k



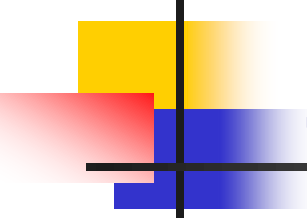
Symmetric Encryption

- ❑ Same key used to encrypt and decrypt
 - Therefore k must be kept secret
- ❑ DES was most used symmetric block-encryption algorithm (created by US Govt)
- ❑ 2001 NIST adopted new block cipher - Advanced Encryption Standard (AES)
 - Keys of 128, 192, or 256 bits, works on 128-bit blocks



Asymmetric Encryption

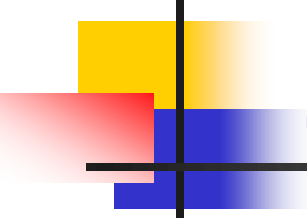
- ❑ Public-key encryption based on each user having two keys:
 - public key – published key used to encrypt data
 - private key – key known only to individual user, used to decrypt data
- ❑ Must be an encryption scheme that can be made public without making it easy to figure out the decryption scheme
- ❑ Most common is RSA
 - No efficient algorithm is known for finding the prime factors of a number
 - k_e is the public key, k_d is the private key
 - N is the product of two large, randomly chosen prime numbers p and q
 - Encryption algorithm is $E(m) = m^{k_e} \bmod N$, where $k_e k_d \bmod (p-1)(q-1) = 1$
 - The decryption algorithm is then $D(c) = c^{k_d} \bmod N$



Symmetric vs. Asymmetric Encryption

- ❑ Symmetric (private) key systems
 - Efficient implementation (GB/sec throughput per CPU core)
 - Difficult key management
- ❑ Asymmetric (public) key systems
 - Much higher computational complexity → Slow implementation
 - Easy key management

Would quantum computing make them useless?



Security in Data Centers

- ☐ Security of in-transit data
- ☐ Security of at-rest data
- ☐ Security of in-use data

In-Transit Data Security

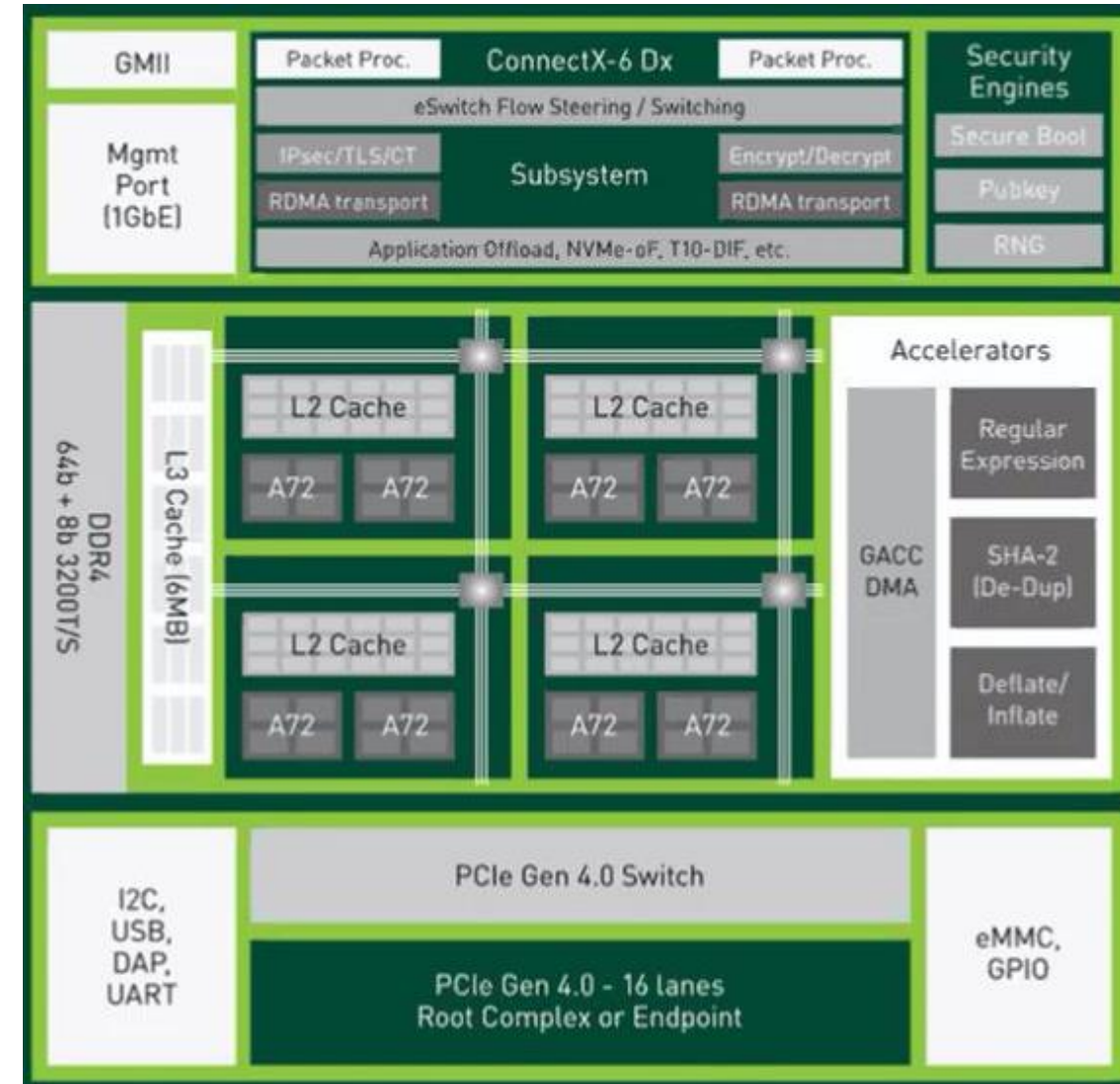
❑ TLS (Transport Layer Security), formerly known as SSL (Secure Sockets Layer), is the most widely used protocol for in-transit data security

- Public key cryptography: Key exchange, authentication
- Private key cryptography: Data transmission

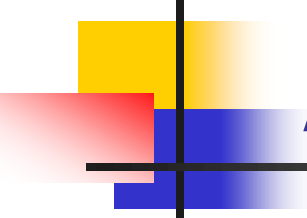
❑ SmartNIC: A natural solution

Key size	1 core	2 cores	4 cores	8 cores	PKA
1024-bit	7900	15273	28411	52581	30986
2048-bit	1591	3115	5722	10554	5239
4096-bit	152	296	537	993	743

of RSA operations per second. PKA refers to the performance measured on Mellanox Bluefield SmartNIC



Nvidia Bluefield DPU

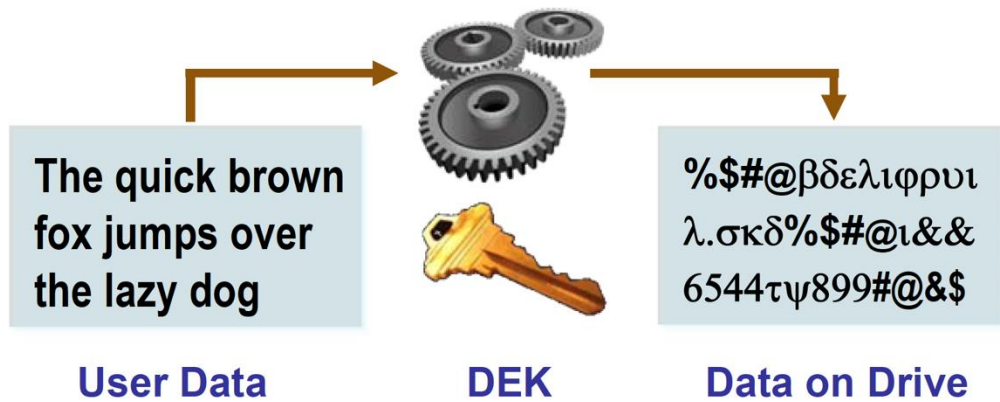


At-Rest Data Security

- ❑ Hardware support: Self-encrypting drive (SED)
- ❑ Standardized by Trusted Computing Group (TCG) <https://trustedcomputinggroup.org>
- ❑ Why self-encrypting drive?
 - Transparency and simplicity
 - Strong security of keys (key generated by true random number generator in drive)
 - Combination with in-storage transparent compression
 - Custom hardware logic for faster, low-power processing
 - Instant, secure data erase
 - Only signed firmware can be loaded into the drive

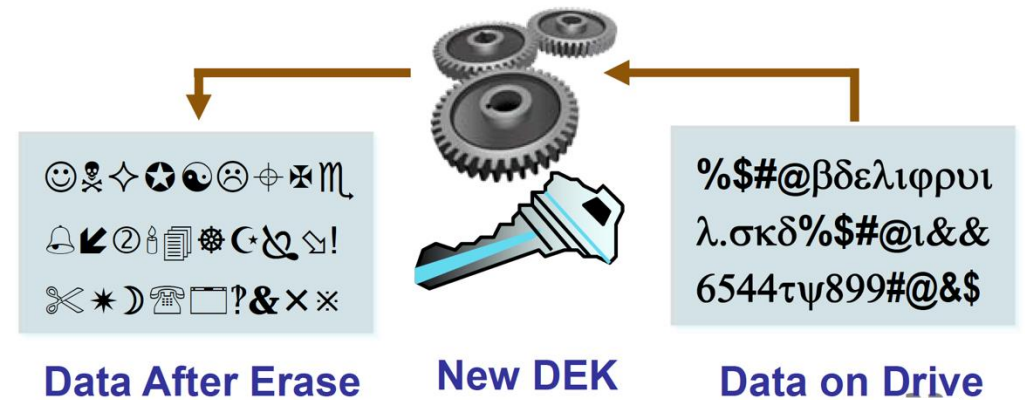
Cryptographic Erase

Encryption Process



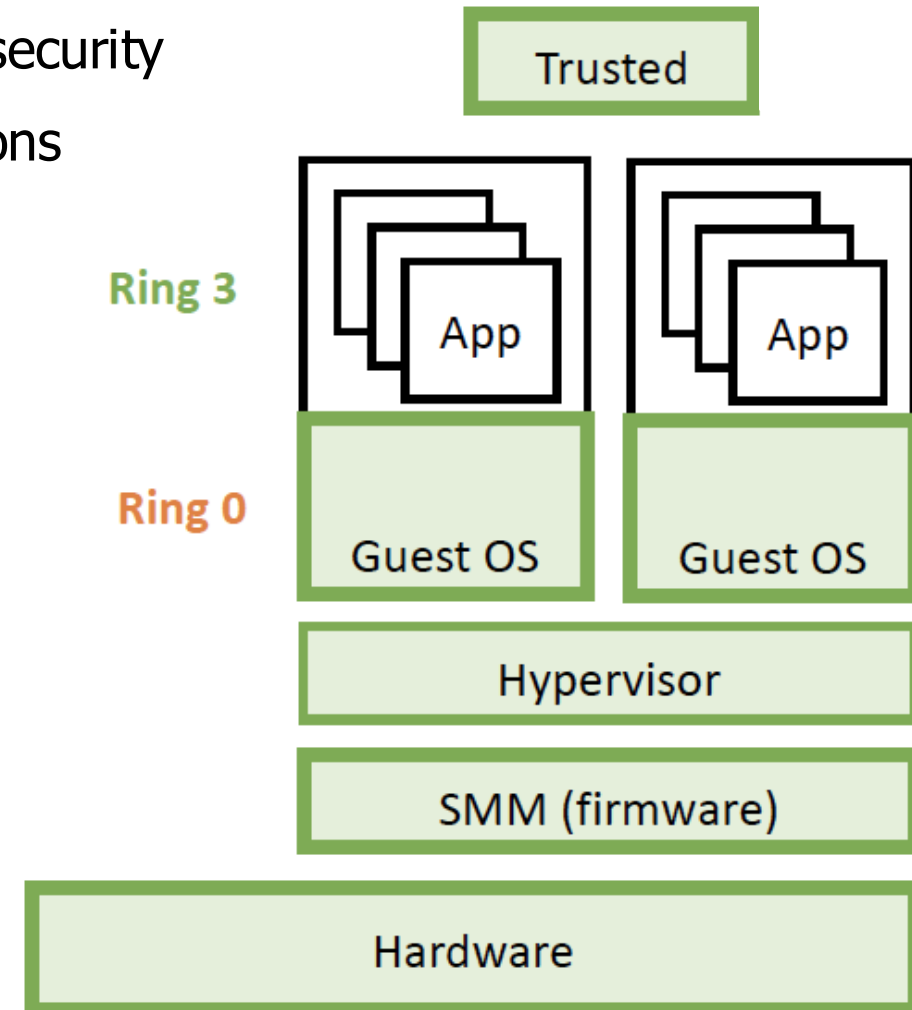
Change DEK

Decryption (After Erase)



In-Use Data Security

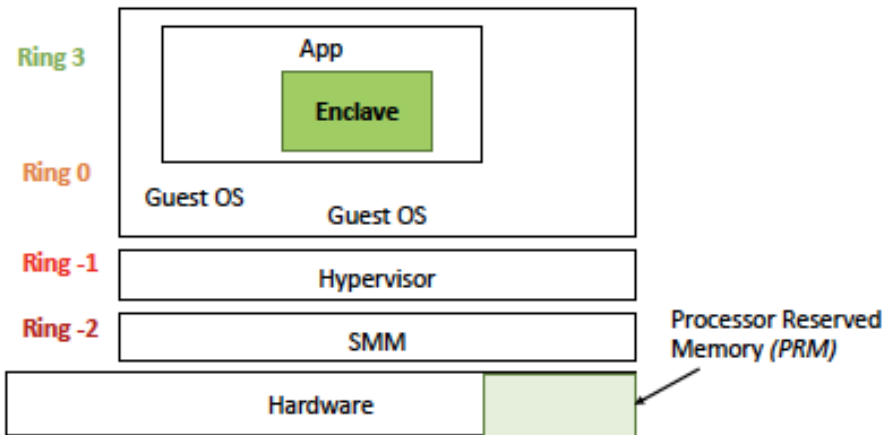
- ❑ A much harder problem compared with in-transit/at-rest data security
- ❑ Traditional practice: Trust everything other than user applications
 - Should we really trust OS, hypervisor, and firmware?
 - Privileged software attack
 - ❖ Directly see the modify application code and data
 - ❖ Mess up with address translation, context switch, interrupts, IOs, ...
- ❑ We must reduce the trusted computing base (TCB)



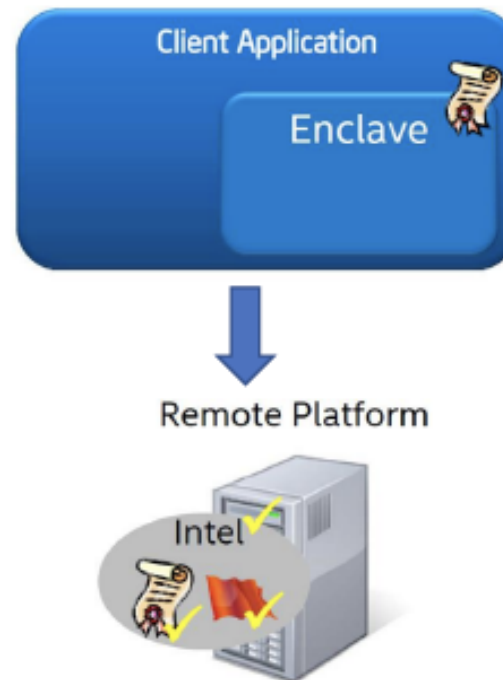
In-Use Data Security

- ❑ Intel SGX for in-use data security

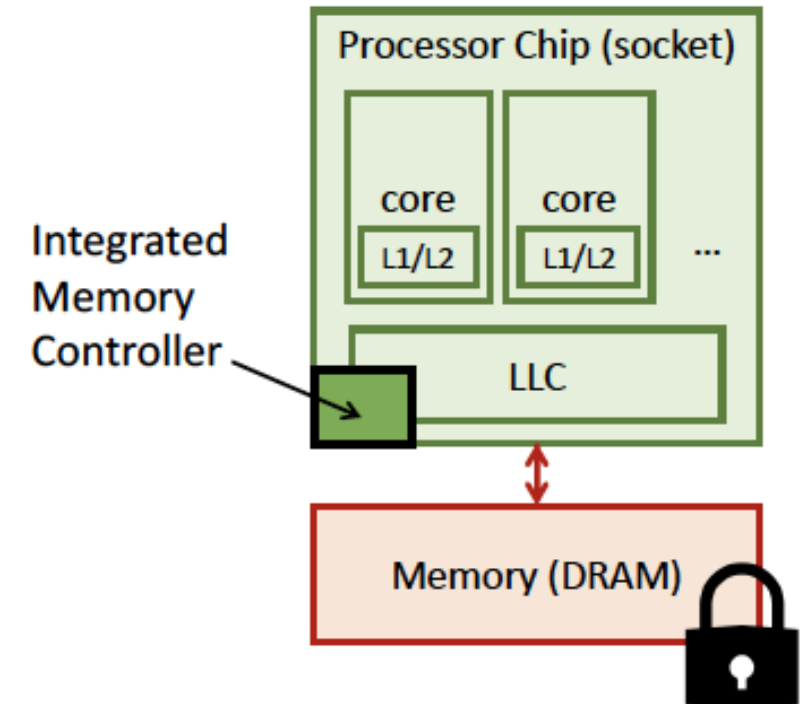
Isolation



Attestation



DRAM Protection

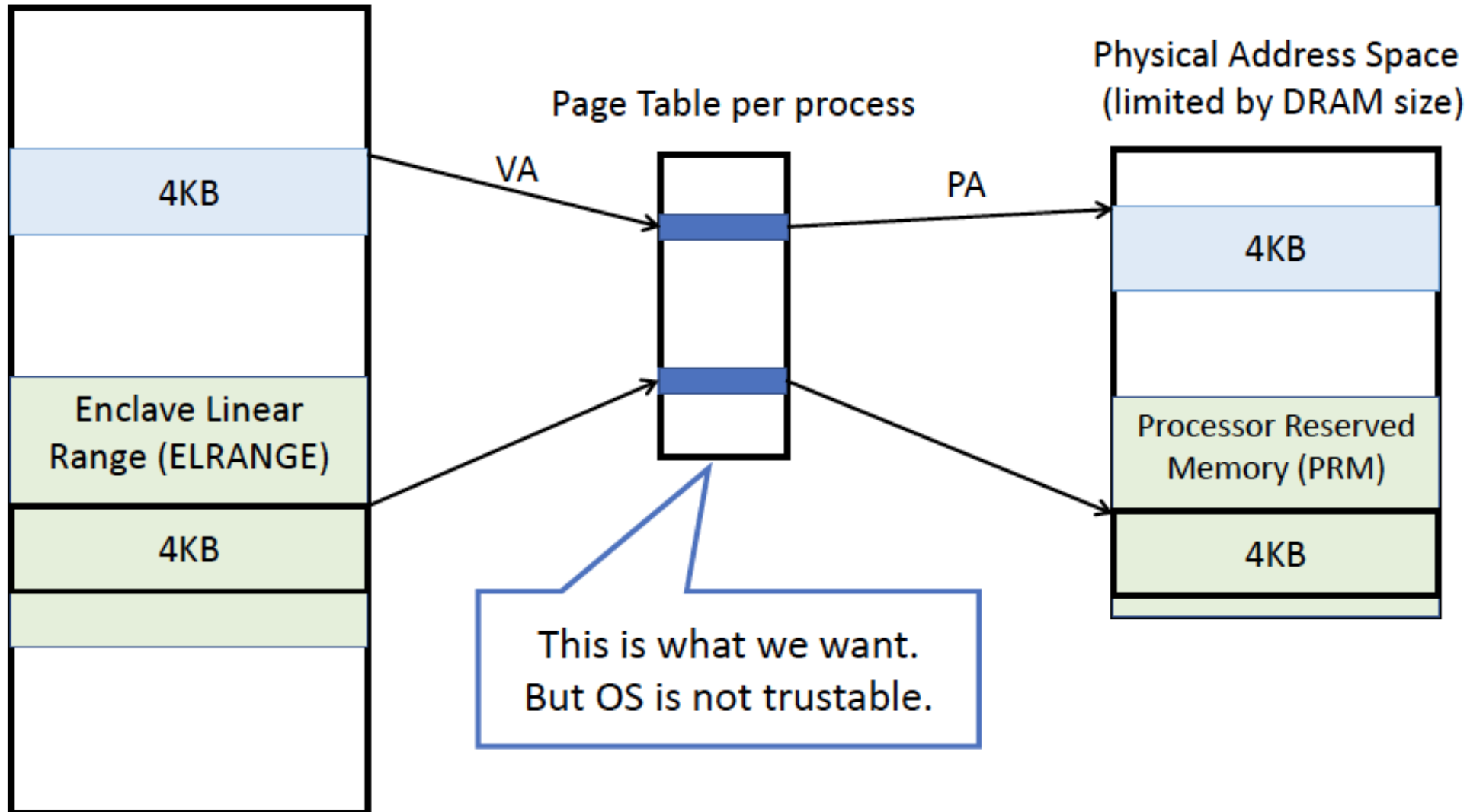


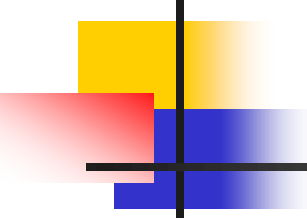
Refer to "Intel SGX Explained" by Victor Costan and Srinivas Devadas for details

Intel SGX

Enhance MMU to ensure enclave → PRM mapping

Virtual Address Space (Programmer's View)





Remarks

- ❑ Security has become the first-class citizen almost everywhere
- ❑ Hardware-based security will play an increasingly important role
 - In-transit data security: SmartNIC
 - At-rest data security: Self-encrypting drive
 - In-use data security: Intel SGX (AMD, IBM, ...)